

Professional Software Architecture Prompt

Act as a **Principal Software Architect, Laravel Core Expert, FinTech Solutions Architect, and Senior Payment Systems Engineer** with extensive experience designing secure, scalable, enterprise-grade payment platforms.

Your task is to design and document a **production-ready, all-in-one Laravel payment package specifically for the Kenyan market**, following modern Laravel standards (Laravel 12+), SOLID principles, Domain-Driven Design (DDD), Clean Architecture, and PSR standards.

The objective is to create a package that serves as the **single payment layer** for any Laravel application, allowing developers to integrate multiple payment providers through one consistent API without changing business logic.

Primary Objectives

Design a reusable Composer package that:

- Supports multiple payment providers through a unified interface.
 - Can be installed into any Laravel project.
 - Is highly modular and extensible using a driver-based architecture.
 - Is production-ready, secure, scalable, well-documented, and thoroughly tested.
 - Supports synchronous and asynchronous payment workflows.
 - Can be extended by third-party developers without modifying the core package.
-

Target Providers

Mobile Money

- M-Pesa Daraja API
- Airtel Money
- T-Kash (optional)

Card Payments

- Stripe
- Pesapal
- Flutterwave
- Paystack

Digital Wallets

- Google Pay
- Apple Pay

Banking

Design an abstraction layer for:

- PesaLink
- EFT
- RTGS
- Virtual Accounts
- Direct Bank APIs

The architecture should make it easy to add additional providers without changing existing application code.

Architecture Requirements

Design the package using:

- Clean Architecture
- Domain-Driven Design (DDD)
- Repository Pattern
- Strategy Pattern
- Factory Pattern
- Adapter Pattern
- Dependency Injection
- Laravel Service Container
- Laravel Manager Pattern
- Laravel Facades
- Contracts/Interfaces
- Service Providers
- Event-Driven Architecture
- Queue-Based Processing

The architecture should strictly follow SOLID principles.

Package Structure

Produce a professional folder structure similar to:

- src/
- Contracts/
- Drivers/
- Managers/
- Services/
- DTOs/
- Actions/
- Events/
- Listeners/
- Jobs/
- Models/
- Enums/
- Exceptions/
- Middleware/
- Traits/
- Http/
- Webhooks/
- Support/
- Facades/
- Config/
- Database/
- Tests/
- Documentation/

Explain the responsibility of every folder and class.

Driver System

Create a driver system similar to Laravel's Cache, Storage, Mail, and Queue managers.

Example:

```
Payment::driver('mpesa');  
  
Payment::driver('airtel');  
  
Payment::driver('stripe');  
  
Payment::driver('pesapal');
```

Every driver must implement a common contract.

Example responsibilities:

- Charge
- Authorize
- Capture
- Verify
- Refund
- Reverse
- Balance
- Transaction Status
- Register Webhooks
- Handle Callbacks
- Validate Requests

Fluent API

Design a fluent API similar to Laravel's Query Builder.

Example:

```
Payment::driver('mpesa')
    ->amount(2500)
    ->currency('KES')
    ->phone('254712345678')
    ->customer($user)
    ->reference('INV1001')
    ->description('School Fees')
    ->metadata([
        'student_id' => 10,
        'invoice' => 455,
    ])
    ->charge();
```

The API should feel natural to Laravel developers.

Unified Payment Response

Every provider should return the same response object regardless of the gateway.

Example methods:

- successful()
- failed()
- pending()
- cancelled()
- transactionId()
- providerReference()
- provider()
- amount()
- currency()
- message()
- status()
- receiptNumber()
- rawResponse()

Database Design

Design migrations for:

- payments
- transactions
- payment_attempts
- refunds
- webhooks
- payment_logs
- provider_accounts
- payment_methods
- audit_logs

Include:

- indexes
- foreign keys
- UUID support
- soft deletes
- timestamps
- JSON metadata

Explain every column.

Payment Features

Support:

- STK Push
 - C2B
 - B2C
 - B2B
 - Card Payments
 - Payment Links
 - QR Payments
 - Bank Transfers
 - Recurring Payments
 - Subscription Billing
 - Split Payments
 - Marketplace Payments
 - Scheduled Payments
 - Partial Refunds
 - Full Refunds
 - Payment Reversals
 - Saved Payment Methods
 - Multi-Currency
 - Multi-Merchant
-

Security

Implement enterprise-grade security.

Include:

- Webhook signature validation
 - HMAC verification
 - OAuth token management
 - Encrypted credentials
 - Idempotency keys
 - Replay attack prevention
 - Request signing
 - Rate limiting
 - Fraud detection hooks
 - Audit logging
 - PCI DSS considerations
 - OWASP best practices
 - Secure secrets management
-

Laravel Features

Integrate with:

- Events
- Queues
- Notifications
- Mail
- Broadcasting
- Cache
- Scheduler
- Artisan Commands
- Validation
- Policies
- Gates
- Middleware
- Eloquent
- API Resources

Events

Create events for:

- PaymentInitiated
- PaymentAuthorized
- PaymentProcessing
- PaymentSuccessful
- PaymentFailed
- PaymentCancelled
- PaymentExpired
- PaymentRefunded
- PaymentReversed
- WebhookReceived
- WebhookProcessed

Explain how developers can listen to these events.

Queues

Support asynchronous processing using Laravel queues.

Include:

- retries
 - exponential backoff
 - dead-letter handling
 - failed jobs
 - retry policies
-

Configuration

Design a publishable configuration file supporting:

- multiple providers
 - environment variables
 - sandbox mode
 - production mode
 - callback URLs
 - retry settings
 - timeout settings
 - queue settings
 - webhook secrets
 - logging options
-

Testing

Provide a complete testing strategy using PHPUnit and Pest.

Include:

- Payment::fake()
 - assertion helpers
 - feature tests
 - unit tests
 - integration tests
 - webhook testing
 - queue testing
 - event testing
 - mock providers
-

Documentation

Generate:

- installation guide
 - configuration guide
 - quick start
 - API documentation
 - driver development guide
 - webhook guide
 - troubleshooting
 - upgrade guide
 - contribution guide
-

Deliverables

Produce a complete technical blueprint that includes:

1. High-level system architecture.
2. Package directory structure.
3. UML/class diagrams (where appropriate).
4. Database schema.
5. Contracts and interfaces.
6. Service layer design.
7. Driver architecture.
8. Event lifecycle.
9. Queue workflow.
10. Authentication flow.
11. Webhook lifecycle.
12. Transaction lifecycle.
13. Error handling strategy.
14. Security architecture.
15. Testing strategy.
16. Performance optimization techniques.
17. Scalability considerations.
18. Example Laravel implementation.
19. Future roadmap.
20. Best practices for open-source package development.

The final output should be written as if preparing a professional architecture specification for an enterprise-grade open-source Laravel package that could become the standard payment solution for the Kenyan ecosystem.